

Pointers (in class)

CSC 240 Data Structures Fall 2025

Marcus Birkenkrahe

October 30, 2025

Contents

1	Objectives	2
2	Getting the Address of a Variable	3
3	PRACTICE Using the address-of operator	4
4	Pointer Variables	5
5	Reference variables	7
6	Pointers vs. references and reason for pointers	8
7	Creating and using pointer variables	10
8	PRACTICE Store integer address in a pointer	10
9	C++urious George	12
10	Indirect access to variables	13
11	Re-pointing pointers	14
12	PRACTICE Working with pointers	15
13	C++urious George	16
14	PRACTICE Pointer challenge	18
15	Arrays and pointers	18

16 Why you can step through arrays using pointers	20
17 PRACTICE Using subscripts with pointers	21
18 Pointer arithmetic	23
19 Initializing pointers	24
20 Pointers as function parameters	25
21 Using pointer parameters to accept array addresses	27
22 PRACTICE Pointers as function parameters	30
23 Dynamic memory allocation	31
24 Dynamically allocating an array	33
25 PRACTICE Dynamically Allocated String Array in C++	34

1 Objectives

We don't assume any knowledge of pointers but if you do have any, it will be useful to review the concept and the practice of pointers.

1. Understand what a pointer is and how it differs from a reference.
2. Use pointers with arrays and functions.
3. Perform basic pointer arithmetic.
4. Dynamically allocate and deallocate memory using raw pointers.
5. Introduce the concept of smart pointers and avoid memory leaks.

All topics:

- ☐ Getting the Address of a Variable
- ☐ Pointer and Reference Variables
- ☐ Relationship between Arrays and Pointers
- ☐ Pointer Arithmetic

- ☐ Initializing Pointers
- ☐ Pointers as Function Parameters
- ☐ Dynamic Memory Allocation

2 Getting the Address of a Variable

- The address-of operator `&` returns the memory address of a variable.
- Where exactly is this "memory address" on the computer?

It's in RAM (Random Access Memory) of the computer, usually somewhere in the "stack" or in the "heap".

- When the variable is undefined, you get any old value:

```
double amount; // double precision floating-point variable

cout << "Address: " << &amount // address-of variable
<< ", value = " << amount // value of variable
<< ", Size = " << sizeof(amount) << " bytes." << endl; // size of variable
```

```
Address: 0x7fff4d9433e0, value = 6.75633e-310, Size = 8 bytes.
```

- Why are the addresses always different?

Because of ASLR (Address Space Layout Randomization) - done for security reasons (harder to inject malicious code in changing memory).

- The following fragment declares three variables.

```
char letter; // 1 byte ( 8 bits)
short number; // 2 bytes (16 bits)
float amount; // 4 bytes (32 bits)
```

- In the computer memory, they may be arranged like this:
- The address itself is represented in hexadecimal (base 16) format. On a 64-bit computer system, the length of an address is typically 64 bits or 8 bytes.

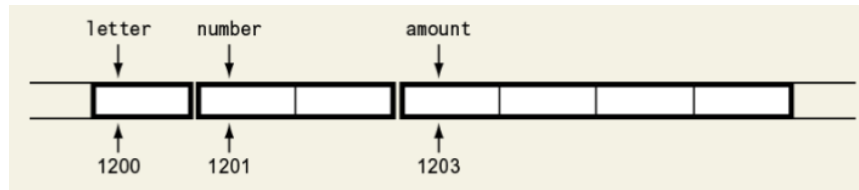


Figure 1: Variables and their addresses (Gaddis fig 9-1).

- How many addresses can be represented on a 64-bit computer system?

2^{64} with 0 or 1 on 64 positions (ca. 18 quintillion) -

- Is each byte in the computer's memory assigned to a unique address?
If so, how would we know this?

Yes, each byte in the memory has a unique physical address.

- Demonstration:

```
char a='A', b='B';
printf("%p\n", (void*) &a); // print address of a using a generic pointer
printf("%p\n", (void*) &b); // print address of a using a generic pointer
```

```
0x7ffe4a494fb6
0x7ffe4a494fb7
```

3 PRACTICE Using the address-of operator

- Write a C++ program that
 1. Declares an integer variable `x` and assigns it the value 25.
 2. Prints the address of `x`, `&x`.
 3. Prints the size of `x` in bytes, `sizeof(x)`
 4. Prints the value stored in `x`.
- Sample output:

```
The address of x is 0x7ffc1970c684
The size of x is 4
The value stored in x is 25
```

- Solution:

```
// This program uses the & operator to determine the address of a
// variable and the sizeof operator to determine its size.
```

```
#include <iostream>
using namespace std;

int main()
{
    int x = 25;

    cout << "The address of x is "      << &x      << endl
         << "The size of x is "        << sizeof(x) << endl
         << "The value stored in x is " << x        << endl;
    return 0;
}
```

```
The address of x is 0x7ffcdaff9664
The size of x is 4
The value stored in x is 25
```

4 Pointer Variables

- A pointer variable is a variable that holds a memory address. When it is used to point at data, its type has to coincide with the type of the data: An integer pointer can point at an integer, etc.
- When an array is passed to a function, it "decays" to a pointer to the first array element.
- What happens in the following code example with regard to variable addresses?

```
#include <iostream>
using namespace std;

void showValues(int [], int); // function prototype

int main()
```

```

{
    const int SIZE = 5;
    int numbers[SIZE] = {1,2,3,4,5}; // initialize array of SIZE elementsx
    showValues(numbers,SIZE); // call function
    return 0;
}

```

```

void showValues(int values[], int size)
{
    for (int count = 0; count < size; count++)
        cout << values[count] << " ";
}

```

1 2 3 4 5

- Explanation:

1. In `main`, the name of the array `numbers` and its `SIZE` are passed to the function `showValues`
2. In the function, the `values` parameter receives the address of the array `numbers` - it points to `numbers[0]`
3. Inside `showValues`, anything done to `values` is done to `numbers`.

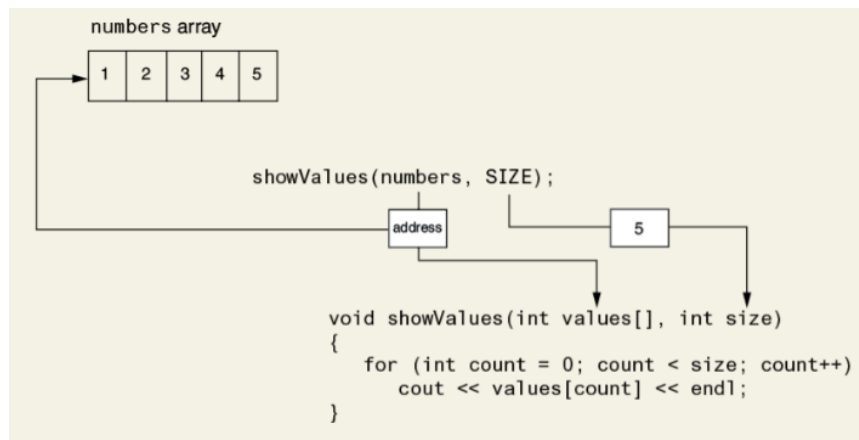


Figure 2: Passing an array to a function (fig. 9-2, Gaddis)

5 Reference variables

- A reference variable acts as an alias for another variable. Anything you do to the reference variable is done to the variable it references.
- What happens in the following code example with regard to variable references?

```
#include <iostream>
using namespace std;

void getOrder(int&); // function prototype

int main()
{
    int jellyDonuts;
    getOrder(jellyDonuts);

    return 0;
}

void getOrder(int& donuts)
{
    cout << "How many donuts do you want? ";
    cin >> donuts;
    cout << "\nOkay, " << donuts << " it is.\n\n";
}
```

```
How many donuts do you want?
Okay, 4 it is.
```

- Explanation:
- Compile and run the code above on the shell.

```
cd ../src
g++ donuts.cpp -o donuts
echo "4" | ./donuts
```

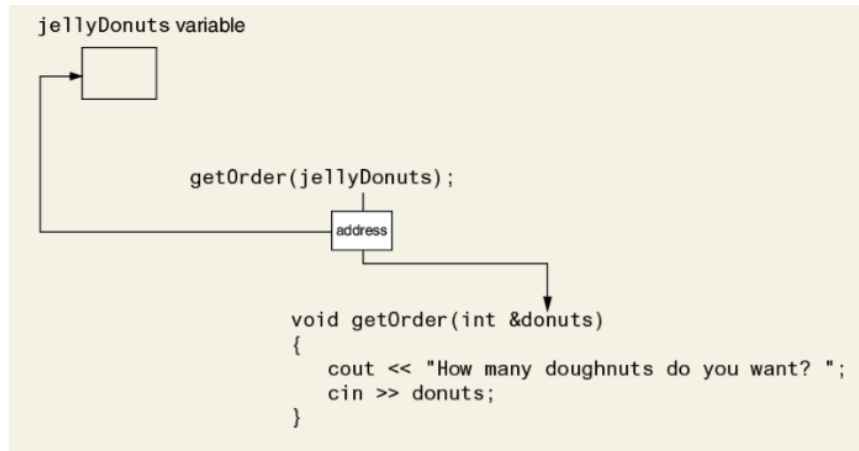


Figure 3: Passing an argument by reference (fig. 9-3, Gaddis)

How many donuts do you want?
 Okay, 4 it is.

– Create input file `../data/donuts.txt`

```

cd ../data
echo "4" > donuts.txt
cat donuts.txt

```

4

6 Pointers vs. references and reason for pointers

- Pointer variables are similar to reference variables but they require more work from you.
- To make a pointer variable reference another item in memory, you have to write code to **fetch** the memory address of that item and **assign** it to the pointer variable (using the address-of operator `&`).
- When you use a pointer to **store** a value in the memory location that the pointer references, you have to specify that it should be stored at a location rather than in the pointer variable itself (using the indirection or dereference operator `*`).

- Programmers use reference variables to make changes permanent between calling routines and functions. But why use pointers?
 1. Pointers are necessary when you want to allocate memory dynamically.
 2. Pointer are useful in algorithms that need to manipulate arrays fast and especially text (stored in strings).
 3. In object-oriented programming (OOP), pointers are useful to create and work with objects and for sharing access.
- Example: Walking through a string using a pointer

```
#include <iostream>

int main() {
    const char* greeting = "Hello, world"; // Declare a C-style string as constant
    std::cout << greeting << std::endl; // Print the whole string.

    // The pointer points at greeting[0].
    std::cout << *greeting << std::endl; // 'H'
    // Moving forward, the pointer points at greeting[1]
    std::cout << *(greeting + 1) << std::endl; // 'e'

    // Traverse the C-style string with a pointer
    const char* p = greeting;
    while( *p != '\0') // check if the char pointed at is \0
    {
        std::cout << *p;
        ++p; // move pointer to next memory cell (byte)
    }

    return 0;
}
```

Hello, world
H
e
Hello, world

7 Creating and using pointer variables

- What does `int* ptr;` mean?
 1. `ptr` is a pointer variable that points to an `int` variable.
 2. `ptr` can hold the value of a memory address (of 4 bytes).
 3. `ptr` at the moment is unassigned (it can point anywhere).
- What's the difference between `int* ptr` and `int *ptr` ?

There is none.

- **Rule:** Always initialize a pointer variable with the `nullptr` - otherwise you're changing a memory address somewhere.

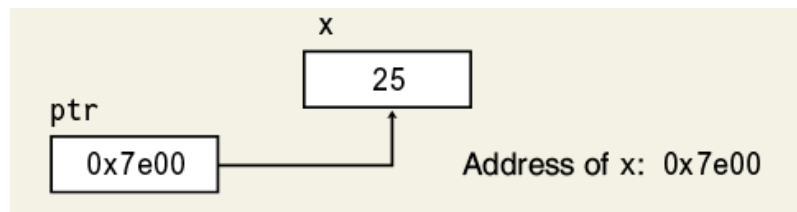
```
int* foo = nullptr; // C++11
int* bar = NULL;    // C or in older C++
```

```
printf("%p %p\n", foo, bar);
cout << foo << " " << bar;
```

```
(nil) (nil)
0 0
```

8 PRACTICE Store integer address in a pointer

- The following program defines an integer variable `x`, assigns a value to it, defines a pointer `ptr` and assigns the null pointer to it. Then the pointer is made to point at the variable address and both the value and the address are printed.



- **Task:** Open the starter code in [OneCompiler.com](https://www.onecompiler.com/), comment it where indicated, and complete the `cout` statements.

- Starter code:

```
// Store integer variable address in a pointer.
#include <iostream>
using namespace std;

int main()
{
    int x = 25;           // Assign value to an integer variable.
    int *ptr = nullptr; // Define pointer that points nowhere in particular
    ptr = &x;             // Store address of integer variable in pointer

    cout << "The value in x is " << *ptr << endl;
    cout << "The address of x is " << ptr << endl;

    return 0;
}
```

The value in x is 25
The address of x is 0x7fff60a94f4c

- Solution (working example with comments)

```
#include <iostream>
using namespace std;

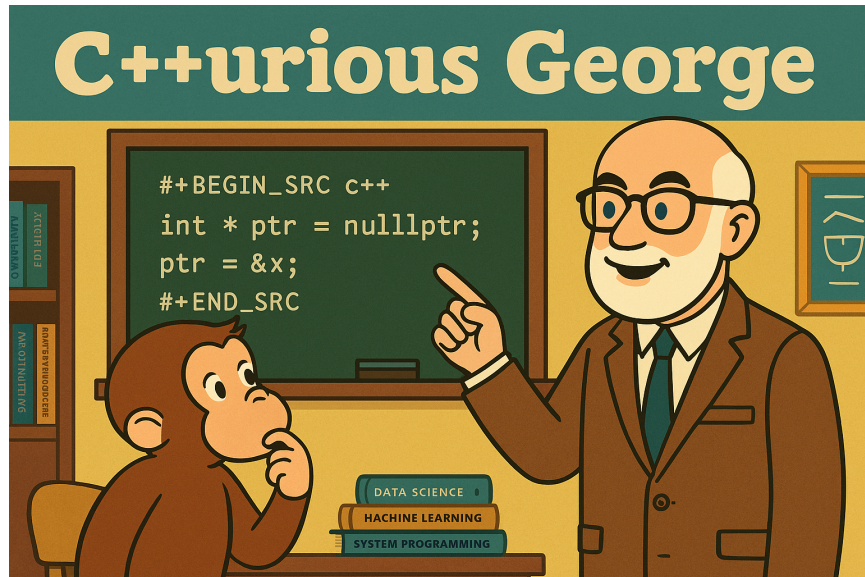
int main()
{
    int x = 25;           // Assign value to integer variable.
    int *ptr = nullptr; // Pointer points nowhere in particular.
    ptr = &x;             // Store address of variable in pointer

    cout << "The value in x is " << x << endl;
    cout << "The address of x is " << ptr << endl;

    return 0;
}
```

The value in x is 25
The address of x is 0x7ffec8b8175c

9 C++urious George



- Can one assign a specific address directly to a pointer?

Yes you can but its dangerous.

```
int* p = (int*)0x7ffee4a2b4fc;
cout << p; // *p will give segmentation fault
*p = 2;
```

- What does the address 0x7e00 tell us about the memory layout of the program (e.g., stack location, variable storage, or memory limits), and how does it relate to the system's overall memory architecture?

It says that the variable or data stored here are located at a specific physical address (32,256th cell?). You'll never see an address like this.

```
echo "ibase=16; 7E00;" | bc
```

```
32256
```

10 Indirect access to variables

- Pointers allow you to indirectly access and modify the variable being pointed to using the **indirection** operator `*` (asterisk).
- Example program:
 1. Assign the address of a variable to a pointer.
 2. Print the value of the variable using the dereferenced pointer.
 3. Assign a value to the location pointed to by the pointer.
 4. Print the value of the variable using the dereferenced pointer.

```
// This program demonstrates the use of the indirection operator.
#include <iostream>
using namespace std;

int main()
{
    int x = 42; // Integer variable
    int *ptr = nullptr; // Pointer variable that can point to an int
    ptr = &x; // Store the address of x in the pointer

    // Use x and ptr to display the VALUE in x
    cout << "Value of x (direct): " << x << endl
         << "Value of x (indirect via pointer): " << *ptr << endl;

    // Assign 99 to the location pointed to by ptr.
    // This will assign 99 to x.
    (*ptr) = 99;

    // Use x and ptr to display the VALUE in x
    cout << "Value of x (direct): " << x << endl
         << "Value of x (indirect via pointer): " << *ptr << endl;

    return 0;
}
```

```
Value of x (direct): 42
Value of x (indirect via pointer): 42
```

```
Value of x (direct):          99
Value of x (indirect via pointer): 99
```

- Explanation:

```
[ptr] ---> addr [x: 42]  // ptr is assigned &x

[ptr] ----*----> 42    // *ptr holds 42

[ptr] ----*----> 99    // *ptr is assigned 99

[ptr] ---> addr [x: 99]  // x holds 99
```

11 Re-pointing pointers

- Since pointers hold memory addresses, they can be re-pointed to other addresses. That happens when you initialize a pointer with the `nullptr` and then assign the address `&x` of an existing variable `x`.
- The following code uses one pointer `ptr` to change the values in three different variables:

```
// Define two integer variables and one integer pointer.
int x = 25, y = 50;
int *ptr = nullptr;

// Display the contents of the integer variables.
cout << "x = " << x << ", y = " << y << endl;

// Use the pointer to manipulate the variables.
ptr = &x; // Store address of x in pointer

(*ptr) += 100; // Add 100 to the value in x

ptr = &y; // Store address of y in pointer

(*ptr) += 100; // Add 100 to the value in y

// Display the contents of the integer variables.
cout << "x = " << x << ", y = " << y << endl;
```

```
x = 25, y = 50
x = 125, y = 150
```

12 PRACTICE Working with pointers

- Complete the starter code in: tinyurl.com/pointer-work

1. Initialize the pointer with the `nullptr`.
2. Assign the address of the variable to the pointer.
3. Change the value of the variable using a pointer.
4. Display the contents of the variable using a pointer.

- Sample output:

The number's value is 99.

- Starter code:

```
// This program demonstrates working with pointers.
#include <iostream>
using namespace std;

int main()
{
    // Assign 42 to the integer 'number'.

    // Assign the null pointer to an integer pointer 'ptr'.

    // Assign the address of number to the pointer.

    // Change value of number to 99 using the pointer.

    // Display the contents of the variable number using a pointer.

    return 0;
}
```

- Solution code:

```
// This program demonstrates working with pointers.
#include <iostream>
using namespace std;

int main()
{
    int number = 42; // Assign 42 to number.

    int *ptr = nullptr; // Assign null pointer to integer pointer.

    ptr = &number;    // Assign address of number to pointer.

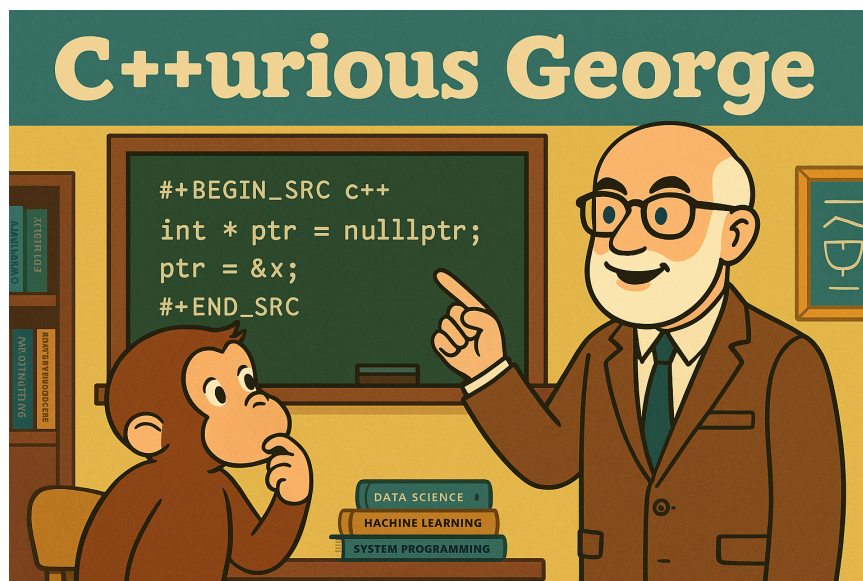
    (*ptr) = 99;      // Change value of number to 99 using the pointer.

    // Display the contents of the variable number using a pointer.
    cout << "The number's value is " << *ptr << ".\n";

    return 0;
}
```

The number's value is 99.

13 C++urious George



- What happens if you assign the address of a non-existing variable to a pointer?

```
int *ptr = num;
```

- A pointer stores the value of an address to a variable. When it is re-pointed so that it stores the value of another variable, what happens to the address of the first variable?

Nothing happens to it - it still exists in memory, only the pointer points elsewhere now. This is different when you use the heap with dynamically allocated (at runtime) data structures: their memory has to be created and deleted.

- Where exactly is the `nullptr` pointing at?

`nullptr` is a special constant - internally it is represented as the address 0x0 (NULL).

- What happens if you try to access or dereference a `nullptr`?

Accessing the value that the Null pointer (doesn't) point at, results in undefined behavior -> segmentation fault.

```
int *ptr = nullptr;
```

```
// forgot to assign a value
cout << *ptr;
```

- If `ptr` is a pointer variable that holds the value of an address, and `*ptr` is the value of that variable, what is `**ptr` and what is `&*ptr` and are these allowed? Test your assumptions.

```
int foo = 42;           // integer variable
int *ptr = &foo;        // pointer to integer (stores address)
int **ptr2 = &ptr;      // pointer to pointer to integer
                        // stores address of the pointer-to-int
cout << ptr << " " << *ptr << " " << &*ptr << endl;
cout << *ptr2 << " " << **ptr2 << endl;
```

```
0x7fff00342784 42 0x7fff00342784
0x7fff00342784 42
```

14 PRACTICE Pointer challenge

To do this practice, open tinyurl.com/pointer-challenge

Write each of the code fragments. Answer questions in comments in the code. At the end, upload the URL of your file to Canvas.

1. Write a statement that defines a pointer variable named `fltPtr`. The variable should be a pointer to a `float`. Be sure to properly initialize the variable.
2. Define an integer variable `foo` and a pointer-to-int variable `ptr`. Write a statement that assigns the address of the `foo` variable to the `ptr` variable.
3. Does `foo` have to have a value for this to work? Write the answer in a comment.
4. Define `ptr` as a pointer variable that points to another variable `foo`. Write a `cout` statement that displays the value that `ptr` points to.
5. List three uses of the `*` (asterisk) symbol in C++. Write the answer in a comment.

15 Arrays and pointers

- What is the relationship between arrays and pointers?

Array names can be used as a constant pointer (that means that you cannot change it to point elsewhere), and pointer can be used as array names.

- Example: What will this program print?

```
short numbers[] {10, 20, 30, 40, 50};  
cout << *numbers << endl;
```

10

- How can one retrieve the entire array `numbers` using a pointer?

```
short numbers[] = {10, 20, 30, 40, 50};
cout << *numbers << " "    // retrieving numbers[0]
<< *(numbers + 1) << " "    // retrieving numbers[1]...
<< *(numbers + 2) << " "
<< *(numbers + 3) << " "
<< *(numbers + 4) << endl;
```

10 20 30 40 50

- Do you think this would also work as a loop?

```
short numbers[] = {10, 20, 30, 40, 50};
for (int i=0; i<5; i++)
    cout << *(numbers + i) << " ";    // same as num[i]
```

10 20 30 40 50

- Would it work as a range-based loop, too?

```
short numbers[] = {10, 20, 30, 40, 50};
for (int num : numbers) // does not know of an i
    cout << *(numbers + i) << " ";    // here i is not declared
```

- There's an even more pointer-ly way to loop since the array name is a constant pointer (to the array's first element):

```
short number[] = {10, 20, 30, 40, 50};
for (short *p = number; p < number + 5; p++)
    cout << *p << " ";
```

10 20 30 40 50

- Can you understand how this `for` loop works?

1. The **start** point(er) is `number[0]` same as `number` and accessed as a dereferenced pointer `*p`.

2. The **stop** pointer(er) checks if the address of **p** is smaller than the address of **number** (**number**[0] offset by five **short** numbers).
 3. The **incrementor** moves the pointer by one address of 2 bytes each.
- How can you be sure that **short** is 2 bytes long?

```
short x;
cout << sizeof(x);
```

2

16 Why you can step through arrays using pointers

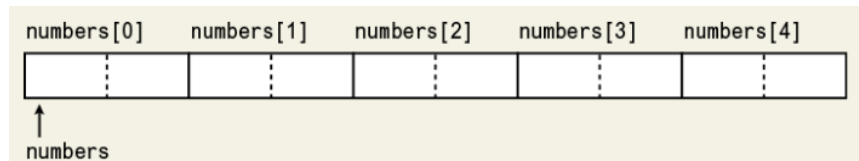


Figure 4: Array elements are stored in contiguous memory locations (Gaddis Fig. 9-5)

When you add a value to a pointer, you are adding that value times the size of the data type being referenced by the pointer - `*(number + 1)` moves 2 bytes forward when **number** is a **short** variable, and 4 bytes when it is an **int** variable: So `*(number + 1)` is actually `*(number + 1 * sizeof(short))` etc.

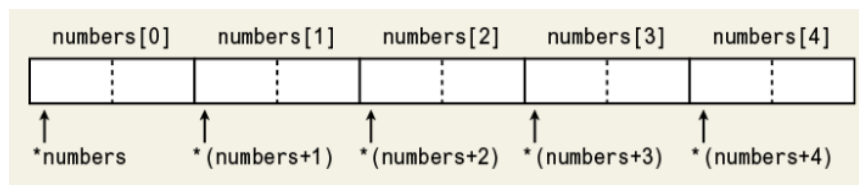


Figure 5: Subscript vs. pointer notation (Gaddis Fig. 9-6)

Length of **short** vs. length of **int**:

```
cout << "Size of short: " << sizeof(short) << " bytes.\n";
cout << "Size of short: " << sizeof(int) << " bytes.";
```

Size of short: 2 bytes.

Size of short: 4 bytes.

But the whole operation works because moving like that through memory makes the pointer point at the next array element.

- **Remember the rule:** `array[index]` is equivalent to `*(array + index)`.

17 PRACTICE Using subscripts with pointers

- This program reveals the deep relationship between arrays and pointers once more. We can come up with four different ways of printing the array elements using pointers.
- In the following program, subscript notation is used with a pointer variable, and pointer notation is used with an array name.
 1. An array of `double coins` is created.
 2. A pointer-to-double `doublePtr` is created.
 3. `coins` is assigned to `doublePtr` (no `&` operator needed!)
 4. The `coins` array is printed using the subscripted `doublePtr`.
 5. The `coins` array is printed using pointer notation.
 6. The `coins` array is printed using pointer arithmetic.
 7. The `coins` array is printed using referencing and dereferencing.

- Sample output:

```
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
```

- Starter code:

```

// Demonstrate connection between arrays and pointers
// Create an array of double variables, and a pointer-to-double
// Display the array elements using pointers in four ways.
// Sample output: 0.05 0.1 0.25 0.5 1
#include <iostream>
using namespace std;

int main()
{
    // Declare array size, array, and pointer
    const int NUM_COINS = 5;
    double coins[NUM_COINS] {0.05, 0.1, 0.25, 0.5, 1.0};
    double *doublePtr = nullptr;

    // Point the pointer at the array.
    doublePtr = coins;

    // Display array using a pointer subscription.
    for (int count=0; count < NUM_COINS; count++)
        cout << doublePtr[count] << " ";

    cout << endl;
    // Display array using pointer notation.
    for (int count=0; count < NUM_COINS; count++)
        cout << *(coins + count) << " ";

    cout << endl;
    // Display array using pointer arithmetic (using addresses)
    for (double *p=coins; p < coins + NUM_COINS; p++)
        cout << *p << " ";

    cout << endl;
    // Display array with referenced array and dereferenced pointers
    for (int count=0; count < NUM_COINS; count++)
    {
        doublePtr = &coins[count]; // store address of element in pointer
        cout << *doublePtr << " "; // dereference the pointer
    }
    return 0;
}

```

```
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
0.05 0.1 0.25 0.5 1
```

18 Pointer arithmetic

- You can change pointer content with addition or subtraction.
- Code along: tinyurl.com/pointer-arithmetic

```
#include <iostream>
using namespace std;

int main()
{
    const int SIZE = 8; // set array SIZE
    int set[SIZE] {5,10,15,20,25,30,35,40}; // create array
    int *numPtr = nullptr; // initialize pointer

    // Make numPtr point to the array.
    numPtr = set;

    // Use the pointer to display the array contents.
    cout << "The numbers in set are:\n";
    for (int count = 0; count < SIZE; count++)
    {
        cout << *numPtr << " "; // print array value pointed at
        numPtr++; // move pointer to next array element
        // same as numPtr = numPtr + 1
    }

    // Display array contents in reverse order.
    cout << "\nThe numbers in set backwards are:\n";
    for (int count = 0; count < SIZE; count++)
    {
```

```

        numPtr--; // move pointer to previous array element
        cout << *numPtr << " "; // print array value pointed at
    }
    return 0;
}

```

The numbers in set are:

```
:5 10 15 20 25 30 35 40
```

The numbers in set backwards are:

```
:40 35 30 25 20 15 10 5
```

- Not all operations are allowed on pointers: ++ and --, +=, -=, are OK, and a pointer may be subtracted from another pointer.

Operation	Example
	int vals[]={4,7,11}; int *valptr = vals;
++, --	valptr++; // points at 7 valptr--; // now points at 4
+, - (pointer and int)	cout << *(valptr + 2); // 11
+=, -= (pointer and int)	valptr = vals; // points at 4 valptr += 2; // points at 11
- (pointer from pointer)	cout << valptr - val; // difference // (number of ints) between valptr // and val

19 Initializing pointers

- You can initialize pointers at definition time:

```

int num, *numPtr = &num;
int val[3], *valPtr = val;

```

- When initializing pointers, you cannot mix data types:

```

double cost;
int *ptr = &cost; // ILLEGAL conversion double* -> int*

```

- You can test if a pointer is properly initialized or not.


```

int *ptr = nullptr;
if (!ptr) // NULL is seen as 0 address by the compiler
    cout << "Pointer is NULL" << endl;
else
    cout << "Pointer is non-NULL" << endl;

int x;
int *ptr2 = &x; // uninitialized pointer not guaranteed to be NULL
if (!ptr2)
    cout << "Pointer is NULL" << endl;
else
    cout << "Pointer is non-NULL" << endl;

Pointer is NULL
Pointer is non-NULL

```

20 Pointers as function parameters

- Pointers can be used as function parameters: This gives the function access to the original argument in the calling function - this is called "pass-by-reference" (address) rather than "pass-by-value" (copy).

```

#include <iostream>
using namespace std;

void increment(int *p) // expects an address
{
    (*p)++; // dereference the pointer and then increment the value
}

int main() {

    int x = 10;
    increment(&x);
    cout << x; // prints 11

    return 0;
}

```

- This is also what using a reference parameter does. What's the difference?
 1. References (&) are aliases for existing variables: no new storage is allocated and you can use the original name in the function.
 2. Pointers require explicit dereferencing (***ptr**) to access or modify the pointed to value.
 3. References can not be reseated to another variable after initialization, unlike pointers.
- **Example:** This program uses two **void** functions that accept addresses of variables as arguments. **getNumber** asks the user for a number as input, and **doubleValue** doubles the input value. Neither function needs a **return** value because of the pass-by-reference.

Code along: tinyurl.com/pointer-parameter - upload to Canvas later.

```
// Ask user for number as input, double the value and print it.
// Uses two void functions that take pointer parameters
// Author: Marcus Birkenkrahe (modified from Gaddis, 9-11, 531)
#include <iostream>
using namespace std;

// Function prototypes
void getNumber(int *); // pass by reference: no need to return anything
void doubleValue(int *);

int main()
{
    int number; // declare an integer

    // Call 'getNumber' and pass the address of 'number'.
    getNumber(&number);

    // Call 'doubleValue' and pass the address of 'number'.
    doubleValue(&number);

    // Display the value in 'number'.
    cout << "That value doubled is: " << number << endl;
```

```

    return 0;
}

// Definition of getNumber. The parameter 'input' is a pointer. The
// function asks the user for a number. The value entered is stored in
// the variable pointed to by 'input'. The calling function's argument
// must be an address.
void getNumber(int *input)
{
    cout << "Enter an integer number: ";
    cin >> *input;
}

// Definition of doubleValue. The parameter 'value' is a pointer. The
// function multiplies the variable pointed to by 'value' by two. The
// calling function's argument must be an address.
void doubleValue(int *value)
{
    (*value) *= 2;
}

```

Enter an integer number: That value doubled is: 61952

```

cd ../src
g++ ptrpar.cpp -o ptrpar # or build it with make
echo "22" | ./ptrpar

```

- What would happen if we would not use `*input` in the `cin` statement but instead `input`?

We would be telling `cin` to store the user input as an address in the pointer variable itself and not in the location it points to. Leads to a type mismatch: `cin` expects an `int&` but `input` is an `int*`.

21 Using pointer parameters to accept array addresses

- The following two (call-by-value) function prototypes are identical:

```
void function(int arr[], int size); // pass by value
void function(int *arr, int size); // pass by reference
```

- Example: Two functions to get sales data and add them up to total sales.
- Code along: tinyurl.com/pointer-array-parameter - upload to Canvas when done.

```
// Get sales data for four quarters, and add up all the sales
// Use pointers as function parameters for getSales and totalSales
// Sample input: 10263.98 12369.69 11542.13 14792.06
// Sample output: The total sales for the year are $48967.86
// Author: Marcus Birkenkrahe (modified from Gaddis 9-12, 529)
#include <iostream>
#include <iomanip>
using namespace std;

// Function prototypes
void getSales(double *, int); // accept pointer to double variable
double totalSales(double *, int); // return the doubled value

int main()
{
    // Declare the sales data array.
    const int QTRS = 4;
    double sales[QTRS];

    // Set the numeric output formatting.
    cout << fixed << showpoint << setprecision(2);
    // Get the sales data for all quarters.
    getSales(sales, QTRS); // not &sales, not *sales
    // Display the totals sales for the year.
    cout << "\nTotal sales for the year are $"
         << totalSales(sales, QTRS) << endl;
    return 0;
}

// Definition of getSales. This function uses a pointer to accept the
// address of an array of doubles. The function asks the user to enter
// sales figures and stores them in the array.
```

```

void getSales(double *a, int s)
{
    for (int count = 0; count < s; count++)
    {
        cout << "\nEnter the sales figure for quarter "
              << (count + 1) << ": ";
        cin >> a[count];
        cout << a[count];
    }
}

// Definition of totalSales. This function uses a pointer to accept
// the address of an array. The function returns the total of the
// elements in the array as a sum.
double totalSales(double *a, int s)
{
    double sum = 0.0;
    for (int count = 0; count < s; count++)
    {
        sum += *a; // a will not work, &a will not work here
        a++; // move down the array
    }
    return sum; // return double total value of all quarters
}

```

```

Enter the sales figure for quarter 1: 10263.98
Enter the sales figure for quarter 2: 12369.69
Enter the sales figure for quarter 3: 11542.13
Enter the sales figure for quarter 4: 14792.06
Total sales for the year are $48967.86

```

- Input data:

```

cd ../data
echo "10263.98 12369.69 11542.13 14792.06" > sales.txt
cat sales.txt

```

```

10263.98 12369.69 11542.13 14792.06

```

22 PRACTICE Pointers as function parameters

- Write a C++ program with a function `increment` that has a pointer-to-integer parameter `ptr`:

1. Define an integer `value` with the value 9.
2. Display `value`.
3. Increment `value` using the function `increment`.
4. Display the value.

- Sample output:

```
9
10
```

- Starter code: tinyurl.com/pointer-parameter-practice

```
// This program increments a set integer value using a function with
// a pointer-to-integer parameter.
#include <iostream>
using namespace std;

// Function prototype
void increment(int *);

int main()
{
    // Define an integer 'value' with the value 9.
    int value = 9;
    // Display 'value'.
    cout << value << endl;
    // Increment 'value' using the function 'increment'.
    increment(&value);
    // Display the value.
    cout << value << endl;

    return 0;
}

// Function definition
```

```
void increment(int *p)
{
    (*p)++;
}
```

9
10

23 Dynamic memory allocation

- Variables may be created and destroyed while a program is running.
- If you know how many variables you need during execution, you can define them up front.
 1. To calculate the area of a rectangle: `length`, `width`, `area`.
 2. To compute payroll for 30 employees: an array `payRoll[30]`.
- When you don't know that, you need to allocate memory on the fly, during program execution:
 1. To store a list of names from a file if you don't know the number of names in the list.
 2. To run an image processing application where you need to store pixel data based on the resolution of images loaded at runtime.
- Most realtime data problems are of the second sort. They require **dynamic memory allocation**, and you need pointers for that.
- How is this done?
 1. When the program is running, it asks the computer to set aside a chunk of memory ("free store", "heap memory")
 2. The heap memory needs to be large enough for specific data type and it needs to provide a starting address.
 3. The starting address is needed - so the program gets a pointer that points to the beginning of that memory.
- In C++, heap memory is requested with the `new` operator:

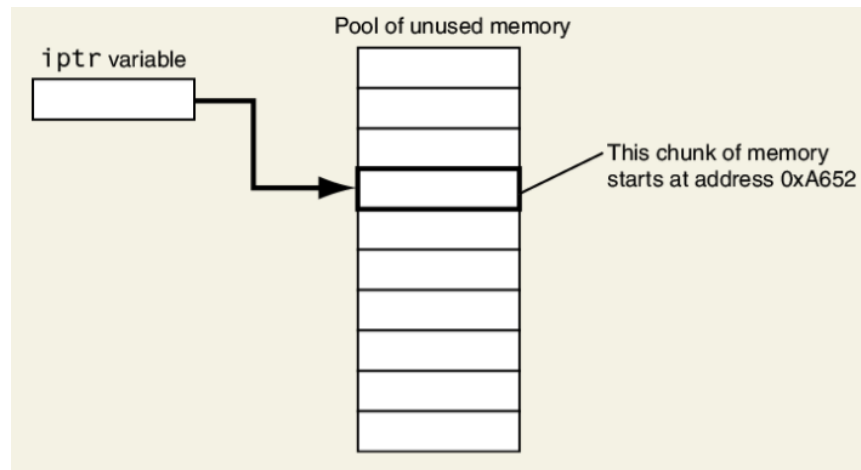


Figure 6: `iptr` points to a chunk of dynamically allocated memory (Fig 9-11).

```
int *iptr = nullptr;
iptr = new int;
```

- To operate on the `new` variable, the dereferenced pointer `*iptr` is used, for example:

```
int *iptr = nullptr; // give me a pointer pointing nowhere
iptr = new int; // make the pointer point a new memory (int)
int total = 100; // loxL variable on the stack
```

```
cout << *iptr << endl; // value at the moment is undefined
cin >> *iptr;
cout << (total += *iptr);
```

```
0
1100
```

- Input

```
cd ../data
echo "1000" > dynamic.txt
cat dynamic.txt
```


1000

- This mechanism is not typically used for single variables.

24 Dynamically allocating an array

- Reserving memory at compile time with a `const SIZE` variable works great as long as you know the length of an array:

```
const int SIZE = 4;
int testScores[SIZE] = {0};
```

- What if we want to ask a user for the number of test scores and we want to store them in an array (without knowing the user's answer)?
- Using the test scores example:

```
// Create a pointer to hold the starter address
int *testScores;
// Create a variable that will hold the size when it is known
int size;
// Get the number of test scores during execution
cin >> size;
// Create the pointer to the array of 'size' elements
testScores = new int[size];
// Use testScores to store test scores
for ( int i=0; i<size; i++)
{
    cout << "\nEnter the score number: " << (i + 1) << " ";
    cin >> testScores[i];
    cout << testScores[i];
}
// Delete the array (using the pointer)
delete [] testScores;
```

```
Enter the score number: 1 95
Enter the score number: 2 78
Enter the score number: 3 88
Enter the score number: 4 100
```

- Failure to deallocate dynamic memory can cause a program to have a **memory leak**. If memory is dynamically allocated in a function without being deleted, the access pointer is lost and the memory can no longer be accessed and just sits there while the program is live.
- Input

```
cd ../data
echo "4" > testScores.txt # size
echo "95 78 88 100" >> testScores.txt
cat testScores.txt
```

```
4
95 78 88 100
```

25 PRACTICE Dynamically Allocated String Array in C++

- Using the starter code below, write a program that asks for the number of participants of an event and reads in their names.
 1. Define a pointer-to-string **names**.
 2. Define an integer variable **count** for the number of participants.
 3. Ask for the number of participants and input **count**.
 4. Output number to check.
 5. Dynamically allocate **new string[count]** memory for **names** (this reserves a **string** array of **count** names).
 6. Loop over **names** and ask for one name after the next (totalling **count**)
 7. Output each name to check.
 8. **delete** the **string** array.
 9. Assign the **nullptr** to the **names** pointer.
 10. Run the program with the names "Alice", "Bob", and "Carol".
- Sample output:

```
Enter the number of participants: 3
Enter name of participant: Alice
Enter name of participant: Bob
Enter name of participant: Carol
```

- Always free memory with `delete[]` to avoid memory leaks.
- Starter code: <https://tinyurl.com/dynamically-allocated>

```
// This program asks the user for the number and the names of
// participants, stores the names in an array, and outputs them.
// header files
#include <iostream>
#include <string>
// namespace directive
using namespace std;
// main program
int main()
{
    // Define a pointer-to-string 'names'.
    string *names = nullptr;
    // Define an integer variable 'count' for the number of participants.
    int count;
    // Ask for the number of participants and input 'count'.
    cout << "Enter number of participants: ";
    cin >> count;
    // Output count to check
    cout << count;
    // Dynamically allocate 'new' 'string[count]' memory for 'names'.
    names = new string[count];
    // Loop over 'names' and ask for one name after the next.
    for (int i=0; i<count; i++)
    {
        cout << "\nEnter name of participant: ";
        cin >> names[i];
        // Output the name after each input to check.
        cout << names[i];
    }
    // delete the string array.
    delete [] names;
```

```
    // Assign the nullptr to the 'names' pointer.  
    names = nullptr;  
  
    return 0;  
}
```

```
Enter number of participants: 4  
Enter name of participant: Alice  
Enter name of participant: Bob  
Enter name of participant: Carol  
Enter name of participant: Joe
```

Sample input: 4 Alice Bob Carol Joe